

# SAÉ2.04-3 : Création d'une infrastructure AD avec Samba4

P. Martini

**Résumé :** Nous avons vu lors du module R1.11 — initiation à l'intelligence artificielle — puis au cours du module R2.02 — machines virtuelles et conteneurisation — et enfin au cours de la SAÉ2.03 — conteneurisation à l'aide de Docker — comment créer et gérer les conteneurs, découvert les solutions permettant de gérer au mieux ces fichiers, et étudié les différences avec la plateforme de virtualisation VMWare.

Au cours du module R1.01c, nous avons mis en place une solution de gestion des clients Windows à l'aide d'un serveur Samba. Cette partie de la SAÉ met en pratique les solutions proposées par Samba pour intégrer les machines Window\$, Mac OS et Linux au sein d'un réseau hétérogène, créant un environnement de partage de fichiers, en utilisant l'efficacité des conteneurs.



## Ça peut vous intéresser...

Bien que les conteneurs soient relativement légers en taille, il sera nécessaire de réaliser régulièrement des sauvegardes de votre travail lors des séances en présentiel. Cela peut se faire à l'aide de la commande `commit` — afin de convertir un conteneur configuré en image — ou de scripts contenant les différentes commandes à réaliser, ou encore en utilisant un fichier `dockerfile` permettant de créer une image configurée, ou enfin en synchronisant deux répertoires avec l'option `--volume`. Cela vous permettra d'optimiser votre temps, notamment lors des changements de salles...

N'hésitez pas à jouer avec les différents paramètres, afin de vous approprier les notions proposées...



## Évaluation...

Vous serez évalué en fonction du travail réalisé, par une présentation finale de l'une des quatre parties de la SAÉ2.04. Un compte-rendu comprenant une copie des différentes commandes **ainsi que de leurs résultats**, un fichier comportant les différentes commandes à mettre en œuvre, une vidéo d'expérimentation, l'enregistrement d'une session de terminal — `script` — vous seront bénéfiques pour faciliter l'évaluation.



## Prise de notes...

Appropriiez-vous au mieux les nombreuses notions et surtout les différents éléments que vous découvrirez par vous-même...



## portainer : une alternative à Docker Desktop...

Une image docker, `portainer/portainer-ce`, disponible sur le hub, permet de gérer au mieux l'environnement Docker. Il est adapté aux trois systèmes d'exploitation principaux que sont Windows, Mac OS et Linux, puisqu'il se lance dans un navigateur Web. Pour créer un container à partir de l'image, copier le code suivant dans un terminal, **en une seule ligne** :

```
docker run -d -p 8000:8000 -p 9443:9443 --name portainer --restart=always -v /var/run/docker.sock:/var/run/docker.sock -v portainer_data:/data portainer/portainer-ce
```

Pour utiliser le container, écrivez `https://localhost:9443` dans un navigateur Web.

## I. SAMBA SUR UN SERVEUR UBUNTU : LES MAINS DANS LE CAMBOUIS



### Samba...

Samba est un service Open Source qui implémente le protocole natif Windows de partage de fichiers. Il est en effet très fréquent d'être confronté à une situation où les machines Linux et Windows doivent communiquer, et la solution la plus simple est alors Samba.

Pour cette section, vous utiliserez Docker, ainsi que l'image `Ubuntu officielle du Hub Docker`. Cette image vous permettra de créer votre serveur Samba, ainsi que votre client de test de configuration. Vous pourrez également utiliser une machine Windows, ou le serveur lui-même dans un premier temps.

**Question 1** Créez un réseau Docker permettant de gérer vos conteneurs. Ce réseau devra être configuré afin d'accueillir plusieurs machines hétérogènes, machine physique ou virtuelle Windows, conteneur...

```
docker network create...
```

**Question 2** Démarrez un conteneur sur ce réseau à partir de l'image ubuntu, en lui attribuant une adresse ip statique. Détaillez les différents paramètres. Ceux ci-dessous ne sont pas tous nécessaires et peut-être en manque-t-il...

```
docker run -it --privileged --hostname nom_host --add-host nom_host:ip --network votre_réseau --ip adresse_ip
```

**Question 3** Installez les paquets samba, smbclient et cifs-utils à votre conteneur **sans utiliser le terminal de la machine**.

```
docker exec -it ubuntu-serveur apt update...
docker exec -it ubuntu-serveur apt install samba smbclient cifs-utils nano net-tools iputils-ping...
```



### Samba, les bases...

Le fichier de configuration de Samba, `/etc/samba/smb.conf`, est composé de sections (dont le nom est entouré de `[]`), elles-mêmes composées d'une suite de paramètres de la forme `nom = valeur`. Chaque section (sauf `[global]`) décrit un *partage*, c'est-à-dire une ressource partagée.

Il existe trois sections spéciales, `[global]`, `[homes]` et `[printers]` dont nous parlerons plus loin. Un *partage* est un répertoire auquel on donne accès, ainsi qu'une description des droits d'accès liés à l'utilisateur de ce service. Le nom du *partage* est le nom qui apparaîtra sur la machine distante, et ne doit pas forcément correspondre au nom du répertoire Linux correspondant. Certains partages peuvent être des services de type invité, c'est à dire des services ne nécessitant pas de mot de passe. Un compte Linux particulier, le `guest` account est alors utilisé pour définir les droits d'accès des usagers concernés. Les services définis par les autres sections requièrent un mot de passe.

Les sections spéciales sont décrites ci-dessous :

1. La section `[global]` contient des paramètres globaux au système, tels le `guest account`, le `workgroup` ou le domaine, ou le niveau d'authentification (paramètre `security`);
2. la section `[home]` permet au serveur de créer des partages automatiques pour connecter des utilisateurs à leur home directory;
3. la section `[printers]` décrit les imprimantes partagées et ne sera pas utilisée lors de cette SAÉ.

Les paramètres les plus utiles sont les suivants :

- `path = chemin-d'accès` : permet de spécifier le chemin d'accès du répertoire sur la machine Linux;
- `read only = yes | no` : permet de spécifier un accès en lecture seule;
- `guest ok = yes | no` : permet de spécifier si l'accès au partage requiert un mot de passe;
- `guest account = ...` : permet de spécifier le compte invité sur la machine Linux. Il doit s'agir d'un compte qui existe déjà sur la machine Linux. Ce paramètre se place dans la section `[global]`;
- `security = share/user` : ce paramètre se place dans la section `[global]`. Sa valeur par défaut est `user`, auquel cas un utilisateur doit toujours s'authentifier avec un login et un mot de passe avant de pouvoir accéder aux services offerts par Samba. Avec l'option `share`, aucun mot de passe n'est demandée à la connexion initiale. L'authentification se fait alors au niveau de chaque partage, selon les paramètres particuliers fixés par ce partage (le paramètre `guest` notamment). `security` peut également prendre les valeurs `domain` ou `server`, qui permettent respectivement d'indiquer que l'utilisateur doit soit s'être authentifié sur le domaine (puis le fonctionnement est similaire à `user`), ou d'indiquer qu'un autre serveur est responsable de l'authentification (puis de nouveau similaire à `user`);
- `browsable = yes/no` : ce paramètre permet de spécifier si le partage doit être visible ou non. Sa valeur par défaut est `yes`. Ce paramètre est notamment utile pour la section `[homes]` (voir plus bas);
- Le paramètre `username map = nom-de-fichier` permet de spécifier un fichier donnant le mapping entre les noms d'utilisateurs Samba et Linux;
- Le paramètre `create mode = mode-octal` permet de spécifier les permissions pour la création de fichiers sur le serveur (exemple : `create mode = 0755`). Un paramètre équivalent, `directory mode`, permet de faire de même pour les répertoires. Pour plus de détails voir `man smb.conf`.

**Question 4** Utilisez les commandes vues dans le TP1 de la ressource R1.01c afin d'approvisionner le serveur Samba.

## II. CAHIER DES CHARGES

Dans une entreprise, on définit trois **groupes** d'utilisateurs : `cmoi`, `ctoi`, `cnous`. Chacun des groupes, comporte des utilisateurs qui sont répartis comme tel :

| Groupes      | <code>cmoi</code> | <code>ctoi</code> | <code>cnous</code> |
|--------------|-------------------|-------------------|--------------------|
| Utilisateurs | <code>moi</code>  | <code>toi</code>  | <code>nous</code>  |
| Répertoires  | <code>amoi</code> | <code>atoi</code> | <code>anous</code> |

Au niveau des droits :

- Le groupe `cmoi` possède les droits de lecture dans `amoi`, `atoi` et `anous`, et d'écriture dans `amoi`;
- Le groupe `ctoi` possède les droits de lecture dans `atoi` et `anous`, et d'écriture dans `atoi`;
- Le groupe `cnous` possède les droits de lecture et d'écriture dans `anous`.

Les trois groupes possèdent un répertoire commun nommé `public`, accessible pour tous en lecture et écriture.

Les groupes `cmoi` et `ctoi` ont un répertoire d'échange commun nommé `amoi-atoi` où `cmoi` possède des droits en lecture et écriture et `ctoi` en lecture. Le groupe `cmoi` gère le répertoire `amoi-atoi`.

Les groupes `cmoi` et `cnous` ont un répertoire d'échange commun nommé `amoi-anous`, où `cmoi` et `cnous` possèdent des droits en lecture et `cmoi` en écriture.

**Question 5** Créez les différents groupes Linux.

```
root@b10edf0657f7:/# samba-tool group add cmoi ctoi cnous
```

**Question 6** Créez les différents comptes linux, en les incorporant dans leur groupe.

```
root@b10edf0657f7:/# samba-tool user create votre_utilisateur_du_domaine --given-name=le_prénom --surname=le_nom --mail-address=votre_utilisateur_du_domaine@domain.org --login-shell=/bin/bash!
```

**Question 7** Créez les différents comptes samba permettant de se connecter aux partages.

```
root@b10edf0657f7:/# samba-tool group addmembers votre_groupe_du_domaine votre_utilisateur_du_domaine
```

**Question 8** Mettez en place la configuration permettant de répondre au cahier des charges : fichier `/etc/smb.conf`, section `[global]`, différents partages...

```
[amoi]
    path = /partage/amoi
    valid users = @cmoi
    write list = @cmoi
```

**Question 9** Testez le fonctionnement à l'aide de la commande `testparm`.

```
root@b10edf0657f7:/# testparm
Load smb config files from /etc/samba/smb.conf
Loaded services file OK.
Server role: ROLE_STANDALONE

Press enter to see a dump of your service definitions

# Global parameters
[global]
    ...
[public]
    ...
[amoi]
    ...
[atoi]
    ...
[anous]
    ...
[amoi-atoi]
    ...
[amoi-anous]
    ...
```

**Question 10** Testez l'accès ainsi que les droits aux différents partages à partir d'un client, à l'aide de la commande `smbclient`.

```
root@b10edf0657f7:/# smbclient //172.17.0.3/amoi -U cmoi
Password for [WORKGROUP\cmoi]:
Try "help" to get a list of possible commands.
smb: \>
```

**Question 11** Testez l'accès ainsi que les droits aux différents partages à partir d'un client, à l'aide de l'interface graphique. Sur Linux, on tapera par exemple `smb://cmoi@172.17.0.4` dans l'explorateur de fichiers.

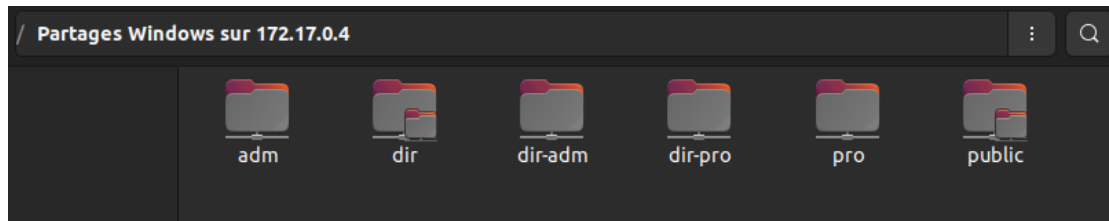


FIGURE 1 – Accès aux partages sous interface graphique

### III. AUTOMATISATION À PARTIR D'IMAGES DOCKER HUB

Pour le moment, votre conteneur a été créé en installant les paquets nécessaires, puis en configurant les répertoires, comptes, droits au sein de la machine.

Des images de machines incorporant Samba existent sur le [hub Docker](#). En voici quelques-unes :

- image Docker Samba incontournable ;
- image Docker Samba configurée en ADDC ;
- image Docker Samba plutôt bien documentée ;
- image Docker Samba configurée avec scripts de création de partages, utilisateurs... ;

**Question 12** En utilisant directement l'image docker proposée par Ahmetozter, mettez en place la même configuration à l'aide des scripts fournis — `shareadd`, `sharelist`, `shareshow`, `sharedel`, `update-samba`, `reload-samba` —. Testez.

**Question 13** Utilisez maintenant celle de David Personette, en créant la ligne de commande permettant de mettre en place les différents éléments demandés. Testez.

```
run -it -p -d --name nom_host --net=host --privileged -hostname nom_host --add-host nom_host:ip -v replocal:repcont \
dperson/samba -p \
    -S -n -w "workgroup" \
    -u "user1;pass1" \
    -u "user2;pass2" \
    -s "public;/partage" \
    -s "share;/srv;no;no;no;no;user1,user2" \
    -s "share1;/partage1;no;no;no;no;user1" \
    -s "share2;/partage2;no;no;no;no;user2"
```

### IV. AUTOMATISATION À PARTIR DE FICHIERS DE CONFIGURATION

**Question 14** Réalisez maintenant l'automatisation de la démarche à l'aide d'un fichier **Dockerfile**, puis **Docker Compose** en suivant l'exemple de sa page github. Testez.